

Course ID: SW 302

User Interface (UI) Development

Dr. Mohamed Sami Rakha

Lecture#5

Cont CSS & Introduction to Tailwind

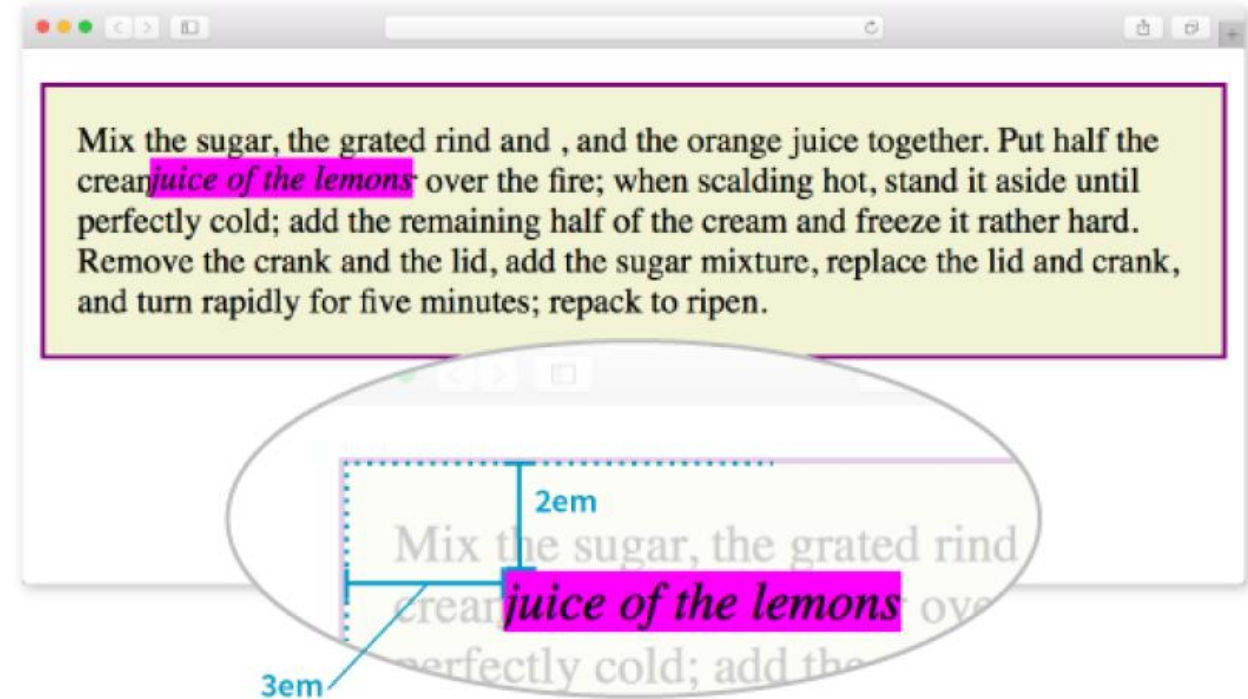
Lecture Outline

Dr. Mohamed Sami Rakha

- **Cont CSS**
- What Tailwind CSS
- Why not just use inline styles?
- How Tailwind Generates CSS?

Containing Blocks Rules

```
p {  
  position: relative;  
  padding: 15px;  
  background-color: #F2F5D5;  
  border: 2px solid purple;  
}  
em {  
  position: absolute;  
  top: 2em;  
  left: 3em;  
  background-color: fuchsia;  
}
```



The relatively positioned `p` element acts as a containing block for the `em` element.

Stacking Order

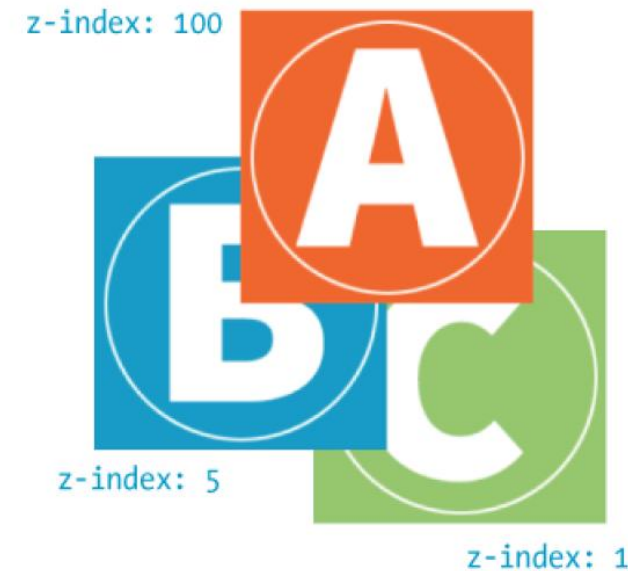
HTML elements overlap (for example, images, boxes, or positioned elements), the stacking order **determines which one appears on top of the others.** Using **z-index** , Values: Number , auto (Default)

```
<style>
div {
  width: 150px;
  height: 150px;
  position: absolute;
}
.red { background: red; top: 20px; left: 20px; z-index: 1; }
.green { background: green; top: 60px; left: 60px; z-index: 2; }
.blue { background: blue; top: 100px; left: 100px; z-index: 3; }
</style>
```

```
<div class="red"></div>
<div class="green"></div>
<div class="blue"></div>
```



By default, elements later in the document source order stack on top of preceding elements.



You can change the stacking order with the z-index property. Higher values stack on top of lower values.

Stacking Order: Example

What is the output of the following code ?

Dr. Mohamed Sami Rakha

```
<style>
div {
  width: 150px;
  height: 150px;
  position: absolute;
}
.red { background: red; top: 20px; left: 20px; z-index: 1; }
.green { background: green; top: 60px; left: 60px; z-index: 2; }
.blue { background: blue; top: 100px; left: 100px; z-index: 3; }
</style>

<div class="red"></div>
<div class="green"></div>
<div class="blue"></div>
```

Stacking Order: Example

What is the output of the following code ?

Dr. Mohamed Sami Rakha

```
<style>
div {
  width: 150px;
  height: 150px;
  position: absolute;
}
.red { background: red; top: 20px; left: 20px; z-index: 1; }
.green { background: green; top: 60px; left: 60px; z-index: 2; }
.blue { background: blue; top: 100px; left: 100px; z-index: 3; }
</style>
```

```
<div class="red"></div>
<div class="green"></div>
<div class="blue"></div>
```



Layout with FlexBox

What is Flexbox

Flexbox is a display mode that lays out elements along one axis (horizontal or vertical).

- Useful for menu options, galleries, product listings, etc.
- Items in a flexbox can expand, shrink, and/or wrap onto multiple lines, making it a great tool for responsive layouts.
- Items can be reordered, so they aren't tied to the source order.
- Flexbox can be used for individual components on a page or the whole page layout.

Flexbox Container

display: flex

Dr. Mohamed Sami Rakha

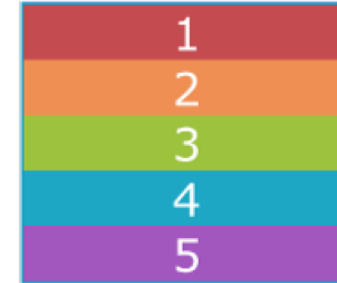
- To turn on Flexbox mode, set the element's **display** to **flex**.
- This makes the element a **flexbox container**.
- All of its direct children become **flex items** in that container.
- By default, items line up in the writing direction of the document (left-to-right rows in left-to-right reading languages).

Flexbox Container

Flexbox Container (cont'd)

ii Rakha

By default, the `divs` display as block elements, stacking up vertically. Turning on flexbox mode makes them line up in a row.

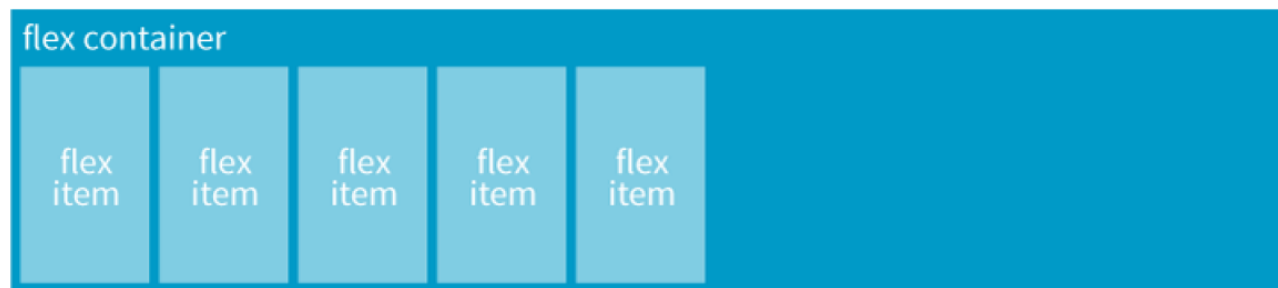


block layout mode

`display: flex;`



flexbox layout mode



Rows and Columns (Direction)

flex-direction

Values: row, column, row-reverse, column-reverse

The default value is **row**(for L-to-R languages), but you can change the direction so items flow in columns or in reverse order:

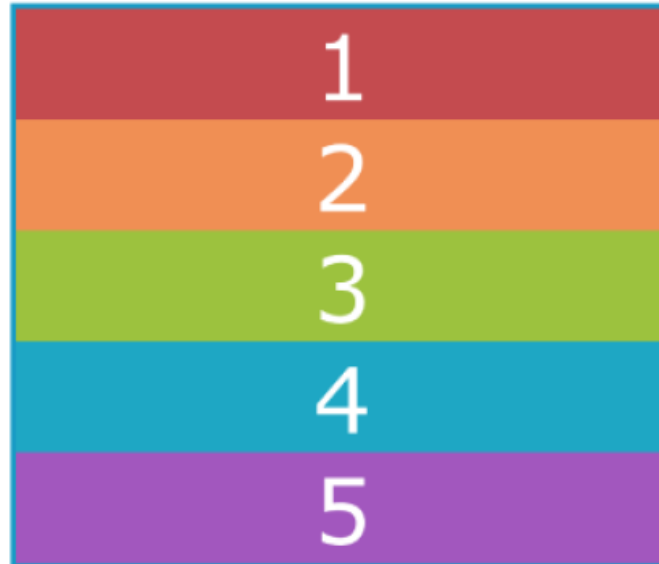
flex-direction: **row**; (default)



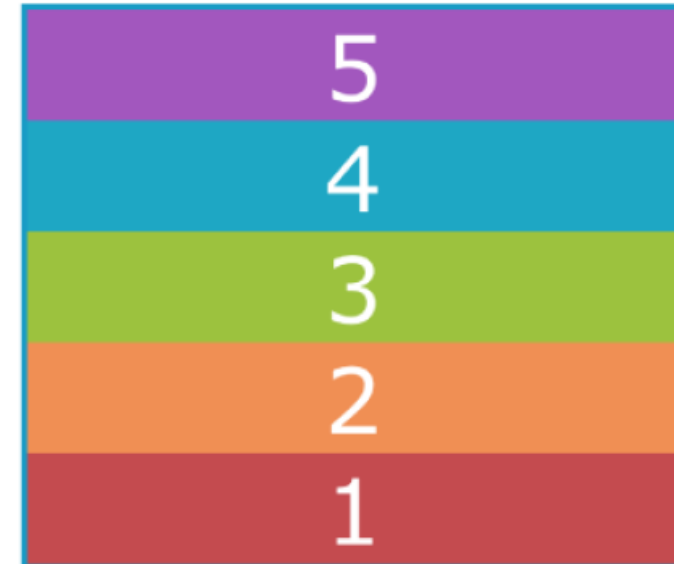
flex-direction: **row-reverse**;



flex-direction: **column**;



flex-direction: **column-reverse**;



Wrapping Flex Lines

flex-wrap

Values: wrap, nowrap, wrap-reverse

Flex items line up on one axis, but you can allow that axis to wrap onto multiple lines with the **flex-wrap** property:

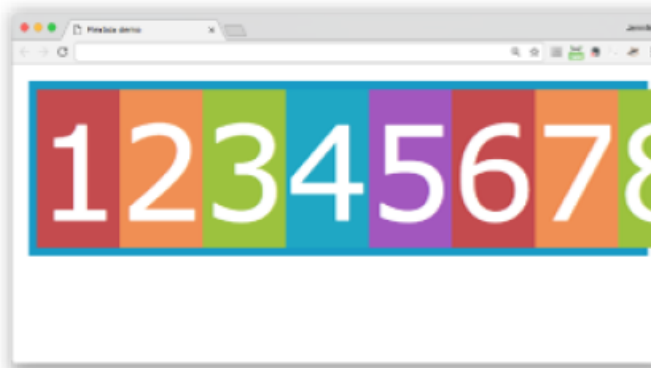
flex-wrap: **wrap**;



flex-wrap: **wrap-reverse**;



flex-wrap: **nowrap**; (default)



When wrapping is disabled, flex items squish if there is not enough room, and if they can't squish any further, may get cut off if there is not enough room in the viewport.

Flex Flow (Direction + Wrap)

flex-flow

Dr. Mohamed Sami Rakha

Values: Flex-direction flex-flow

The shorthand **flex-flow property** specifies both direction and wrap in one declaration.

Example

```
#container {  
  display: flex;  
  height: 350px;  
  flex-flow: column wrap;  
}
```

Aligning on the Main Axis (cont)

When the direction is row , and the main axis is horizontal

`justify-content: flex-start;` (default)



`justify-content: flex-end;`



`justify-content: center;`



`justify-content: space-between;`



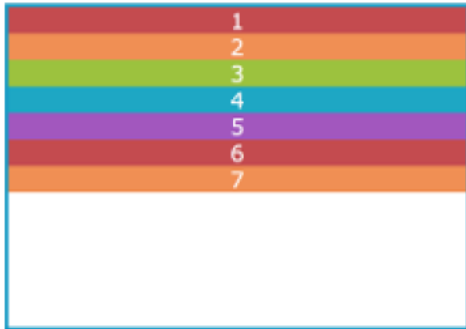
`justify-content: space-around;`



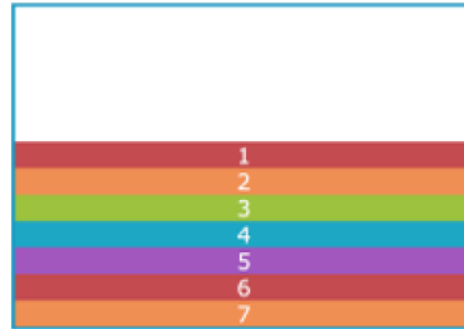
Aligning on the Main Axis (cont)

When the direction is column , and the main axis is vertical

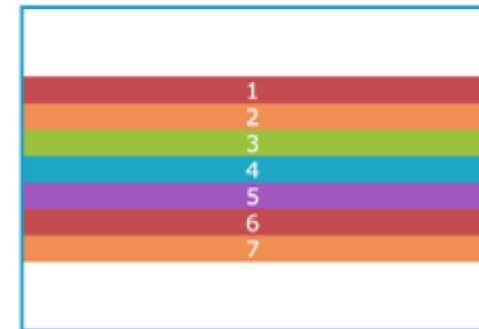
`justify-content: flex-start;` (default)



`justify-content: flex-end;`



`justify-content: center;`



`justify-content: space-between;`



`justify-content: space-around;`

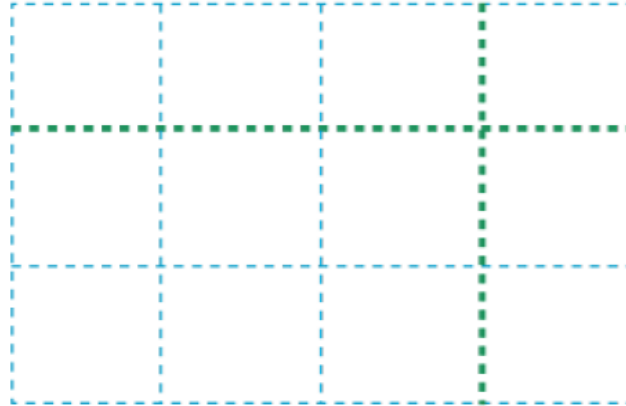


Layout with Grid

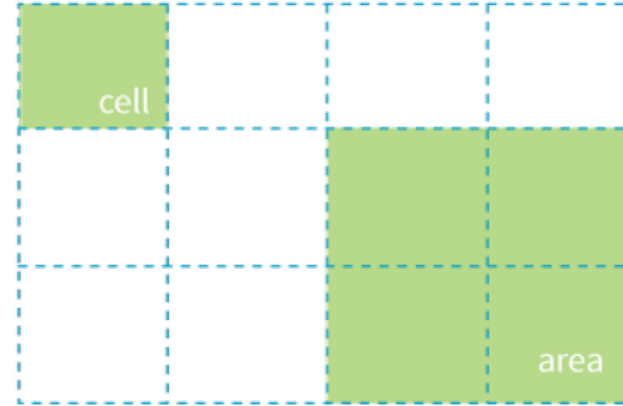
How CSS Grids Work

1. Set an elements **display** to **grid** to establish a grid container. Its children become grid items
2. Set up the columns and rows for the grid (explicitly or with rules for how they are created on the fly).
3. Assign each grid item to an area on the grid (or let them flow in automatically in sequential order).

Grid Terminology

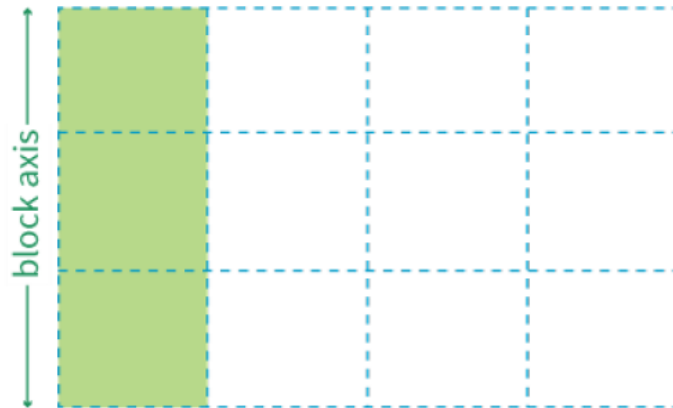


grid lines

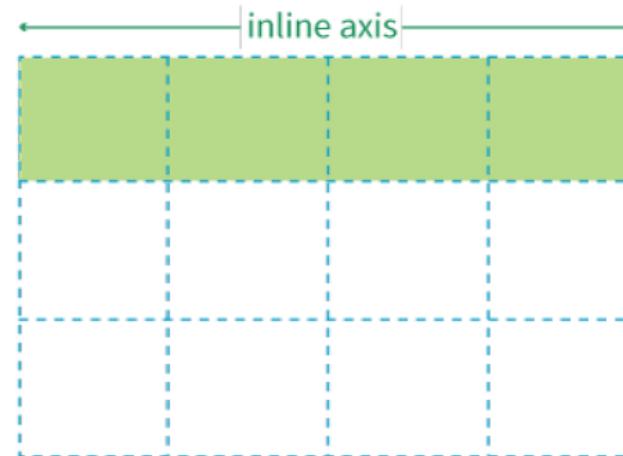


grid cell and grid area

i Rakha



grid track (column)



grid track (row)

Creating a Grid Container

To make an element a **grid container**, set its **display** property to **grid**. All of its children automatically become grid items

The markup

```
<div id="layout">
  <div id="one">One</div>
  <div id="two">Two</div>
  <div id="three">Three</div>
  <div id="four">Four</div>
  <div id="five">Five</div>
</div>
```

The styles

```
#layout {
  display: grid;
}
```

Defining Row and Column Tracks

grid-template-rows
grid-template-columns

Mohamed Sami Rakha

Values: none, list of track sizes, and optional line names

- The value of **grid-template-rows** is a list of the heights of each row track in the grid.
- The value of **grid-template-columns** is a list of the widths of each column track in the grid.

```
#layout {  
  display: grid;  
  grid-template-rows: 100px 400px 100px;  
  grid-template-columns: 200px 500px 200px;  
}
```

The **number of sizes provided determines the number of rows/columns in the grid**. This grid in the example above has 3 rows and 3 columns.

Responsive Web Design

Responsive web design (RWD)

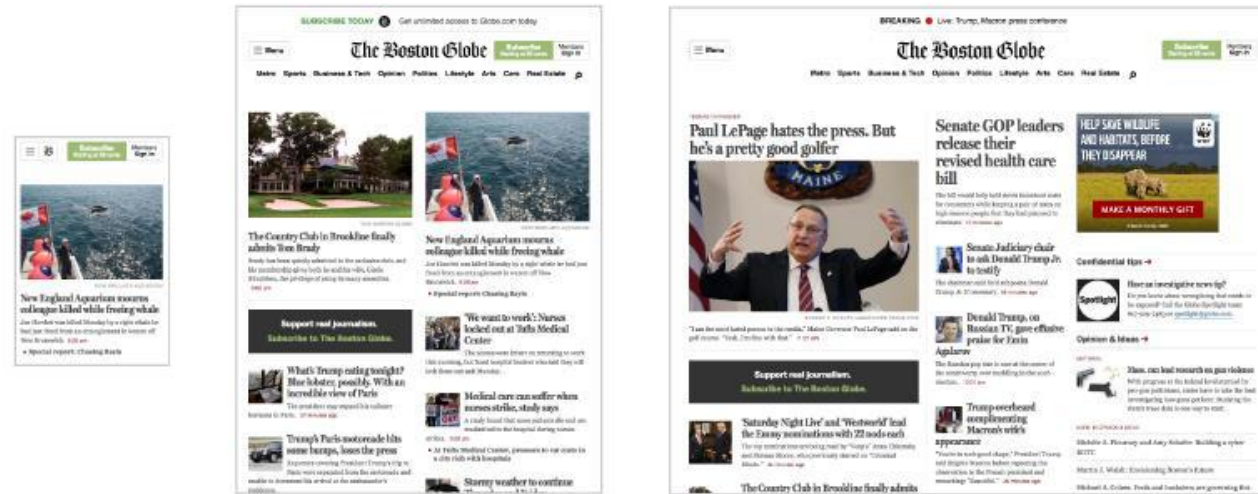
RWD is a design and production approach that allows a web page to adapt to the screen it is viewed on.

The same HTML source is styled differently based on the width of the screen.

It ensures mobile users get the same content as other users, with a presentation that optimizes usability.

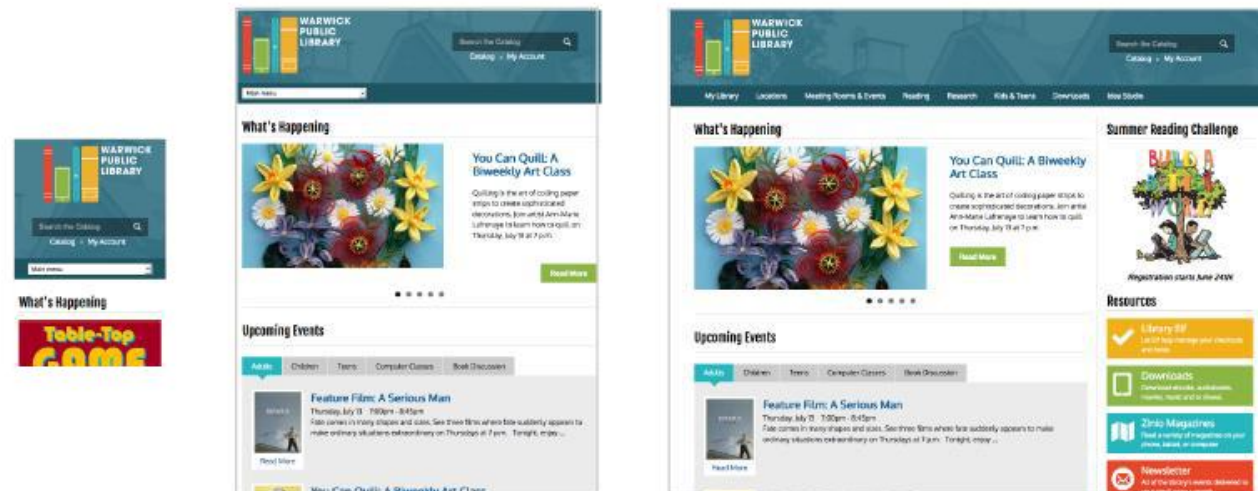
Responsive web design (RWD)

The Boston Globe (*bostonglobe.com*)



ami Rakha

Warwick Public Library (*warwicklibrary.org*)



The Components of RWD

RWD defines the **three core components** as follows:

- **A flexible grid:** Elements are sized so they squeeze and flow into the available browser space.
- **Flexible images:** Images and other embedded media scale to fit their containing elements.
- **CSS media queries:** These are a way to test for browser features such as viewport width and deliver styles accordingly.

Setting the Viewport

- Mobile browsers render pages on a canvas called the **viewport** and then shrink it to fit the screen (**device width**).
- For example, a web page be rendered on a **980-pixel** viewport and shrunk down to fit a **480-pixel** wide screen.
- **To make the viewport the same size as the device width:**

```
<meta name="viewport"  
      content="width=device-width, initial-scale="1">
```

Setting the Viewport

By default, the viewport shrinks to the size of the screen.



With the viewport meta tag, the viewport is created at the same size as the screen.

```
<meta name="viewport"
      content="width=device-width,
              initial-scale=1">
```

viewport = 320pt



d Sami Rakha

Flexible Grids (Fluid Layout)

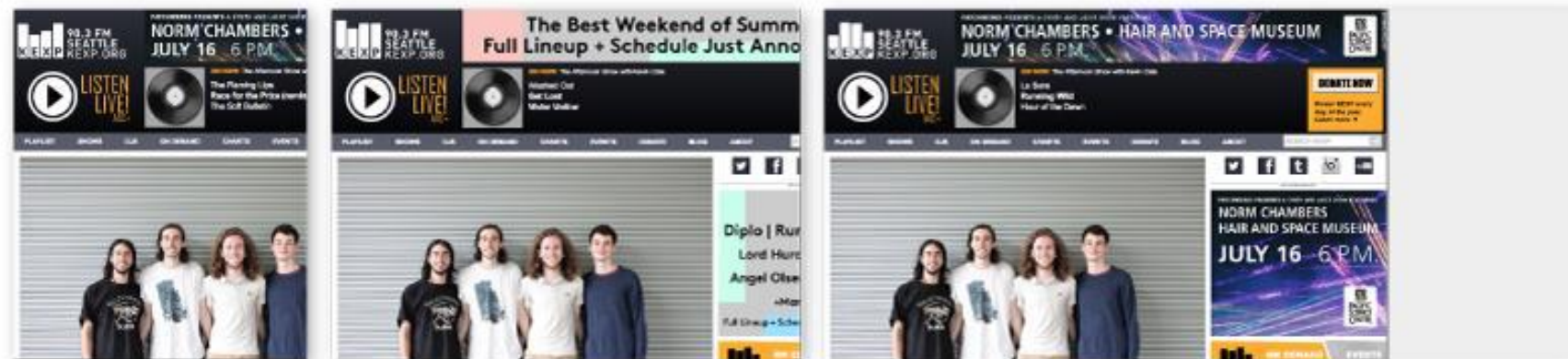
Fluid layouts fill the viewport proportionally.

akha



w3c.org

Fixed layouts stay the same size and may get cut off or leave extra space.



kexp.org

Flexible Grid (Fluid Layout)

Fluid layouts are the best approach for designing for a wide range of screen widths. A fluid layout **adjusts automatically as the viewport width changes, instead of having fixed pixel widths.**

- For flexibility, many methods, use:

- **fr** and **minmax()** and content-based grid track values
- **flex** in Flexbox
- percentage measurements

Example#1: Using CSS Grid for Fluid Layouts

Use **fr** units. The fr unit represents a fraction of available space in a grid container.

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr; /* 3 equal columns */  
  gap: 1rem;  
}
```

As the screen resizes, each column grows or shrinks proportionally.

Example#2: Using Flexbox for Fluid Layouts

Flexbox automatically adjusts elements' **widths** and **spacing**.

Explanation:

- 1 1 200px means:
 - Start each item at 200px,
 - Allow them to grow and shrink.
- flex-wrap: wrap ensures items wrap to the next line when space runs out.

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 1rem;  
}  
  
.flex-item {  
  flex: 1 1 200px; /* flex-grow, flex-shrink, flex-basis */  
}
```

Result → Flexible number of items per row depending on screen size.

Making Images Flexible

To make images scale down to fit the size of their container:

```
img { max-width: 100%; }
```

In this book, Philadelphia Ice Creams, comprising the first group, are very palatable, but expensive. In many parts of the country it is quite difficult to get good cream. For that reason, I have given a group of creams, using part milk and part cream, but it must be remembered that it takes smart "juggling" to make ice cream from milk. By far better use condensed milk, with enough water or milk to rinse out the cans.

Ordinary fruit creams may be made with condensed milk at a cost of about fifteen cents a quart, which, of course, is cheaper than ordinary milk and cream.



If pure raw cream is stirred rapidly, it swells and becomes frothy, like the beaten whites of eggs, and is "whipped cream." To prevent this in making Philadelphia Ice Cream, one-half the cream is scalded, and when it is very cold, the remaining half of raw cream is added. This gives the smooth, light and rich consistency which makes these creams so different from others.

The time for freezing varies according to the quality of cream or milk or water; water ices require a longer time than ice creams. It is not well to freeze the mixtures too rapidly; they are apt to be coarse, not smooth, and if they are churned before the mixture is icy cold they will be greasy or

In this book, Philadelphia Ice Creams, comprising the first group, are very palatable, but expensive. In many parts of the country it is quite difficult to get good cream. For that reason, I have given a group of creams, using part milk and part cream, but it must be remembered that it takes smart "juggling" to make ice cream from milk. By far better use condensed milk, with enough water or milk to rinse out the cans.

Ordinary fruit creams may be made with condensed milk at a cost of about fifteen cents a quart, which, of course, is cheaper than ordinary milk and cream.



If pure raw cream is stirred rapidly, it swells and becomes frothy, like the beaten whites of eggs, and is "whipped cream." To prevent this in making Philadelphia Ice Cream, one-half the cream is scalded, and when it is very cold, the remaining half of raw cream is added. This gives the smooth, light and rich consistency which makes these creams so different from others.

In this book, Philadelphia Ice Creams, comprising the first group, are very palatable, but expensive. In many parts of the country it is quite difficult to get good cream. For that reason, I have given a group of creams, using part milk and part cream, but it must be remembered that it takes smart "juggling" to make ice cream from milk. By far better use condensed milk, with enough water or milk to rinse out the cans.

Ordinary fruit creams may be made with condensed milk at a cost of about fifteen cents a quart, which, of course, is cheaper than ordinary milk and cream.



If pure raw cream is stirred rapidly, it swells and becomes frothy, like the beaten whites of eggs, and is "whipped cream." To prevent this in making Philadelphia Ice Cream, one-half the cream is scalded, and when it is very cold, the remaining half of raw cream is added. This

Media Queries

A media query is a special rule that tests for certain conditions before applying the styles it contains:

```
@media type and (feature: value) {  
    /* styles for browsers that meet this criteria */  
}
```

A few query examples:

- Is it on a screen or printed?
- Is the browser at least a certain width?
- Is the orientation landscape or portrait?

Media Query Example

```
@media screen and (orientation: landscape) {  
  body {  
    background: skyblue;  
  }  
}  
@media screen and (orientation: portrait) {  
  body {  
    background: coral;  
  }  
}
```



When the viewport is in portrait mode, the background color is "coral."



When the viewport is in landscape mode, the background color is "skyblue."

Media Types

- The first part of a media query identifies the **media type**
- The current media types are all , print, screen, and speech

Example:

```
@media print {  
    /* print-specific styles between the brackets */  
}
```

Media Type	Applies To	Typical Use
all	All devices	Default fallback
print	Printers, print the page preview	Print-optimized layout
screen	Displays (desktop, mobile, etc.)	Responsive design
speech	Style for screen readers	Accessibility control

Media Feature Queries

The **media feature queries** test for a particular feature of a viewport or device.

Example:

`h1` headings display in Lobster font only when the viewport is *at least 40em wide*.

```
h1 {  
  font-family: Georgia, serif;  
}  
@media screen and (min-width: 40em) {  
  h1 {  
    font-family: 'Lobster', cursive;  
  }  
}
```



NOTE: `min-width` and `max-width` queries are most useful for RWD.

Breakpoints

A **breakpoint** is the point at which we use a media query to introduce a style change.

- Best practice is to create breakpoints for individual parts of the page rather than the entire page at once.
- Designers generally create the narrow-screen version first and then change components as needed as the screen grows larger.
- **Em-based** breakpoints introduce layout changes proportional to font size.

Breakpoints



At the **480-pixel breakpoint**, the category navigation changes from a list to photos. “Register” is added to the top navigation bar.



At **501 pixels**, “Sell” becomes “Sell on Etsy” (a very subtle adjustment). You can also see more links in the navigation bar under the search field.



At **640 pixels**, the “How Etsy Works” images and messages move above the categories. In smaller views, they were accessible via the “Learn how Etsy works” link in a yellow bar.



Content Hierarchy

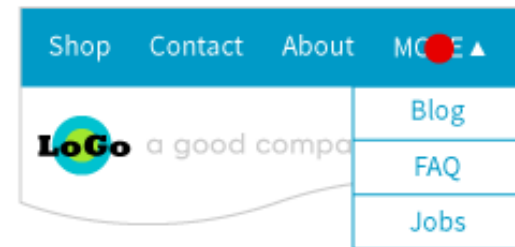
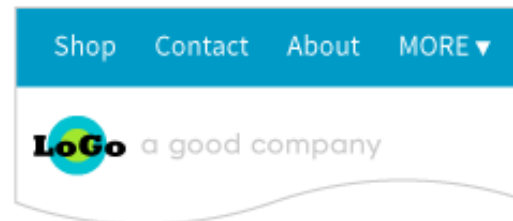
- The **biggest challenge** is deciding what is visible in the **first screen** on small devices.
- You should strive for content parity across devices. Everyone should have access to the same information, even if it s not in exactly the same place.
- Content organization is the most important part of site planning. It s often handled by information architects or content strategists.

Navigation Options

Top navigation



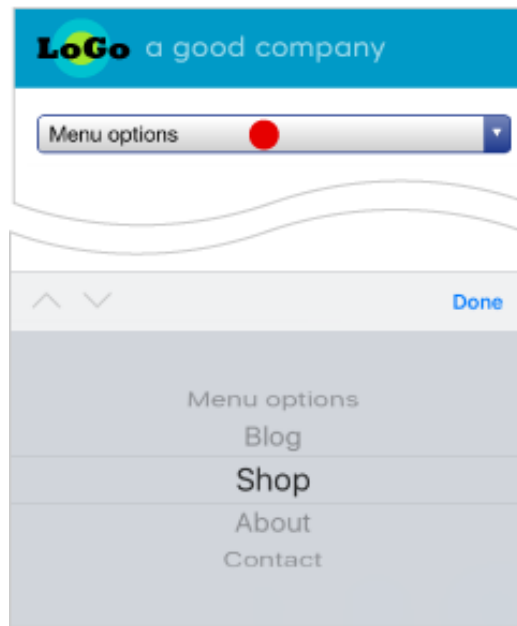
Priority +



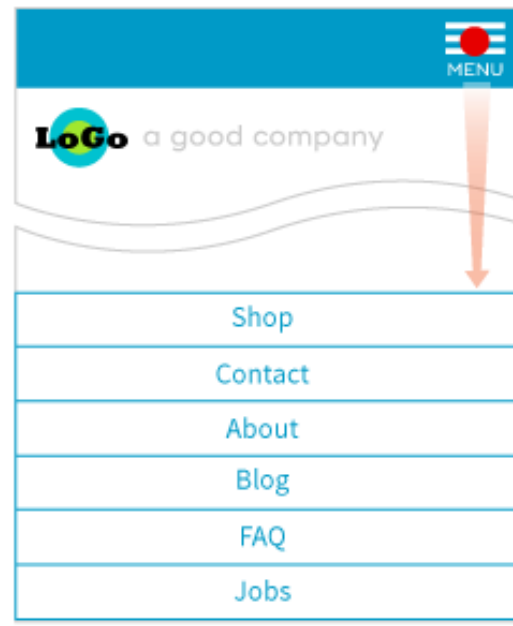
KEY



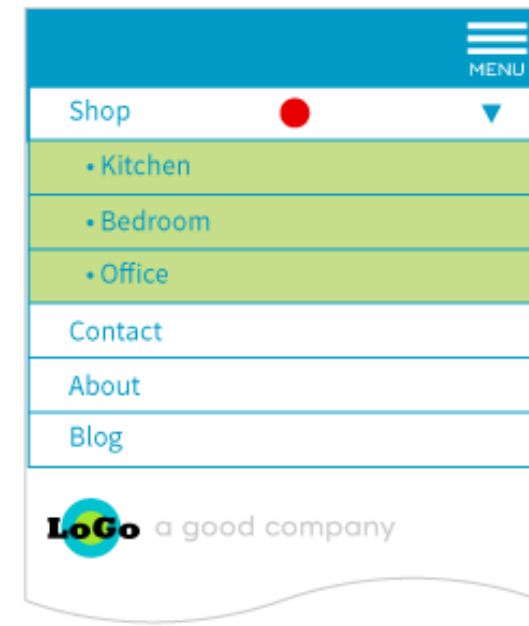
Select menu



Link to footer menu



Accordion sub-navigation

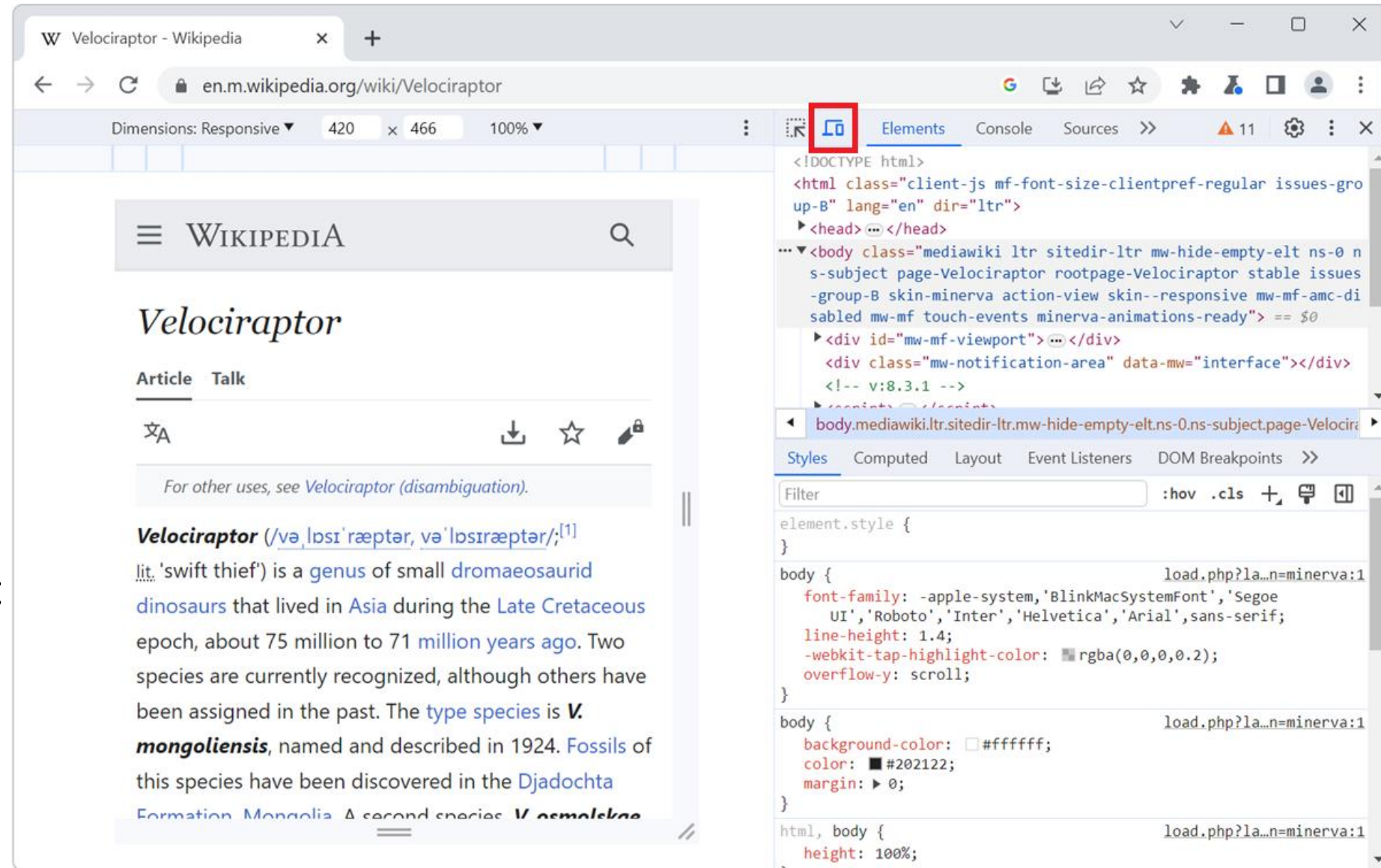


Site Testing

Testing sites in a wide variety of browsing environments is critical:

- **Real devices:** the best way to test is on actual smartphones, tablets, etc.

- **Emulators:** Use a desktop application that simulates device hardware and software.



CSS comments `/*...*/`

```
/* This is a comment.  
It can span many lines in the CSS file. */  
p {  
  color: red; background-color: aqua;  
}
```

CSS

- CSS (like HTML) is usually not commented as rigorously as programming languages such as Java
- The `//` single-line comment style is NOT supported in CSS
- The `<!-- ... -->` HTML comment style is also NOT supported in CSS

We had a good journey with CSS3

Dr. Mohamed Sami Rakha



What Tailwind CSS

What Tailwind CSS

Tailwind CSS is a utility-first CSS framework. That means instead of writing your own CSS rules (like `.btn { ... }`), you use **small, reusable utility classes** directly in your HTML to style elements.



tailwindcss

Unlike other **frameworks**, like Bootstrap, it does not provide a series of **predefined** classes for elements such as buttons or tables. Instead, it creates a list of "**utility**" CSS classes that can be used to style each element by **mixing and matching**.

Try Playing Online:



tailwindPLAY <https://play.tailwindcss.com/>

Philosophy from which Tailwind was created and its basic usage!

Utility classes

The utility-first concept refers to the main differentiating feature of Tailwind. Instead of creating **classes around components** (button, panel, menu, textbox ...), classes are **built around a specific style element** (yellow color, bold font, very large text, center element...). Each of these classes is called a **utility class**.

Each **Tailwind class** (m-4, p-4, bg-yellow-200, font-bold, ...) **represents one CSS rule**. You combine them like building blocks; that's why it's called utility-first.

Example: yellow notice

Result	Please be careful when feeding the birds.	
Code	<pre><div class="m-4 p-4 bg-yellow-200 font-bold rounded-lg"> <p>Please be careful when feeding the birds.</p> </div></pre> 	
Classes	Tailwind	CSS equivalent
	m-4	margin: 1rem;
	p-4	padding: 1rem;
	bg-yellow-200	background-color: rgb(254 240 138);
	font-bold	font-weight: 700;
	rounded-lg	border-radius: 0.5rem;

How Tailwind Works Internally

1- Tailwind has a prebuilt design system, a giant list of CSS utilities like:

- **Colors** (text-gray-700, bg-blue-600).
- **Spacing** (p-4, m-2, px-6).
- **Layout** (flex, grid, justify-center).
- **Typography** (font-bold, text-xl).
- **Borders, shadows, transitions**, etc.

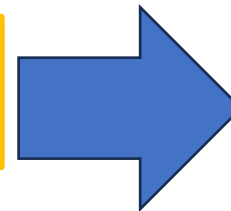
2- When you use these in your HTML, Tailwind's compiler converts them into actual CSS.

When you use the **build** Tailwind:

- Scans your HTML files
- Keeps only the classes you actually use
- Removes all unused ones (so your CSS file is small and fast)



```
<div class="bg-red-500 p-4 text-white">Hello</div>
```



```
.bg-red-500 { background-color: #ef4444; }  
.p-4 { padding: 1rem; }  
.text-white { color: #fff; }
```

How Utility Classes Help!?

1. No Need to Invent New Class Names

Dr. Mohamed Sami Rakha

Utility classes in Tailwind (like p-4, text-center, bg-blue-500) are small, single-purpose classes that each apply one specific style rule. Instead of writing your own CSS selectors for everything, you **compose your design directly in HTML** using these utilities.

```
<div class="hero-section">
  <h1 class="hero-title">Welcome</h1>
</div>
```

```
<style>
.hero-section {
  background: #1d4ed8;
  padding: 4rem;
  text-align: center;
}
.hero-title {
  color: white;
  font-size: 3rem;
}
</style>
```



```
<div class="bg-blue-600 p-16 text-center">
  <h1 class="text-white text-5xl">Welcome</h1>
</div>
```

Benefit:

No **"naming headache."** You don't have to think of names like hero-section, hero-title, banner-container, etc. You just describe what it looks like.

How Utility Classes Help!?

2. Faster Development

Dr. Mohamed Sami Rakha

You build layouts **instantly**, without constantly jumping between HTML and CSS files.



Before:

Write HTML.
Switch to CSS file.
Create selectors.
Switch back to see results.
Repeat 20 times.

You stay in one file,
change a class name, save,
and see it update instantly.

How Utility Classes Help!?

3. Consistent Design System

Dr. Mohamed Sami Rakha

Tailwind enforces consistency automatically because it gives you a predefined design scale:



Padding: p-1, p-2, p-4, etc. (no random p: 13px)

Colors: blue-500, gray-600, red-700, etc.

Font sizes: text-sm, text-lg, text-3xl

The entire project uses the same spacing, fonts, and color logic, with no accidental design drift. Your UI will look professional and consistent without a designer watching every pixel.

How Utility Classes Help!?

4. No CSS Conflicts

Dr. Mohamed Sami Rakha

In big projects, normal CSS often “fights itself”:

```
.card { padding: 10px; }  
.card img { width: 100%; }
```



In Tailwind, styles belong **only to the element** where you use them, no shared global rules that cause bugs. Like no more “why did this section break when I changed that other CSS?”

How Utility Classes Help!?

5. Readability for Developers

Dr. Mohamed Sami Rakha

The Tailwind HTML becomes **self-descriptive**, extremely powerful in team projects.



Even without opening any CSS file, *you instantly know:*

- Background is white
- Has shadow and rounded corners
- Padding 1.5rem
- Text centered

How Utility Classes Help!?

6. Responsive and Interactive by Design

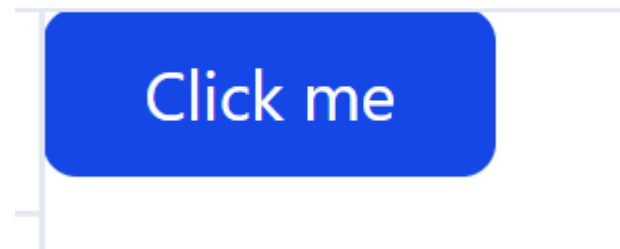
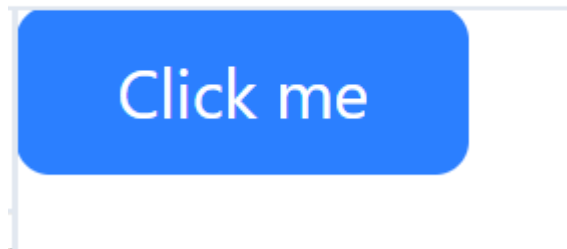
Dr. Mohamed Sami Rakha

Tailwind makes responsive and hover states effortless:



```
<button class="bg-blue-500 hover:bg-blue-700 text-white px-4 py-2 rounded-lg sm:px-6 lg:px-8"> Click me </button>
```

hover:bg-blue-700 affect with hovering on the button

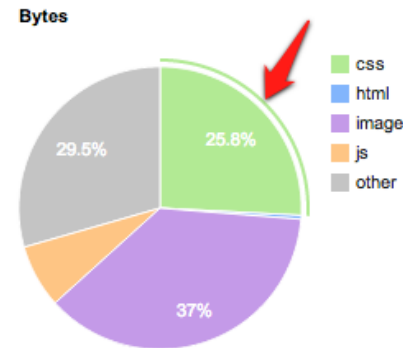


How Utility Classes Help!?

7. Smaller Final CSS (in production builds)

Dr. Mohamed Sami Rakha

When you build with Tailwind's CLI, it scans your HTML and removes all unused classes, so your final CSS is tiny, in comparison to a typical 100–200KB Bootstrap or custom CSS file.



MIME Type	Bytes
image	389564
other	209840
css	271582
js	76104
html	4634
flash	0
text	0

Why not just use inline styles?

Why not just use inline styles?

`<p style="color: red; font-size: 18px;">Hello</p>`

1] Simple, but they cause ****serious problems**** as your project grows.

No Reusability:

If you want to style 50 elements the same way, you'd have to repeat all the styles 50 times.
- That means more typing, more file size, and more maintenance.

With CSS or Tailwind: You define or reuse one class, and apply it anywhere: HTML

`<p class="text-red-500 text-lg">Hello</p>`

Hello

```
.text-red-500 {  
  color: var(--color-red-500) /* oklch(63.7% 0.237 25.331) =  
  #fb2c36 */;  
}
```


Why not just use inline styles?

2] Hard to Maintain: If your client or team wants a color change (say, from red to blue), you'd have to manually update every inline style. **With external CSS or Tailwind**, you change it once (**in CSS or config**) and it updates everywhere.

3] No Responsive Design: Inline styles don't handle responsiveness, you can't do:

style="font-size: 14px on **small screens** and 18px **on large screens**"

Tailwind supports breakpoints directly:

```
<p class="text-base md:text-lg lg:text-xl">Responsive Text</p>
```

Why not just use inline styles?

4. No State Styles (Hover, Focus, Active): Inline styles can't handle dynamic states easily:

```
<p style="color: red;">Hover me</p> <!-- no hover possible -->
```

Tailwind makes it easy:

```
<p class="text-red-500 hover:text-blue-500">Hover me</p>
```

Tailwind supports breakpoints directly:

```
<p class="text-base md:text-lg lg:text-xl">Responsive Text</p>
```

5. Performance and Optimization: Browsers can **cache external CSS files**, improving performance (Generated by **Tailwind**). While, for inline styles, inline styles are re-downloaded and re-rendered each time, slower for large pages.

How Tailwind Generates CSS?

How Utility Classes Help!?

Tailwind does not ship with all possible CSS upfront (that would be huge!). Instead, it **generates only the CSS you actually use** based on your project content.

[1] You write HTML (or JSX, etc.) using Tailwind utility classes, e.g.:

```
<button class="bg-blue-500 text-white px-4 py-2 rounded">Click Me</button>
```

[2] Tailwind CLI scans your source files (HTML, JS, TS, etc.) to find all class names.

This is done via the `content` paths in your **tailwind.config.js**

```
content: ["./src/**/*.html", "./src/**/*.js"],
```

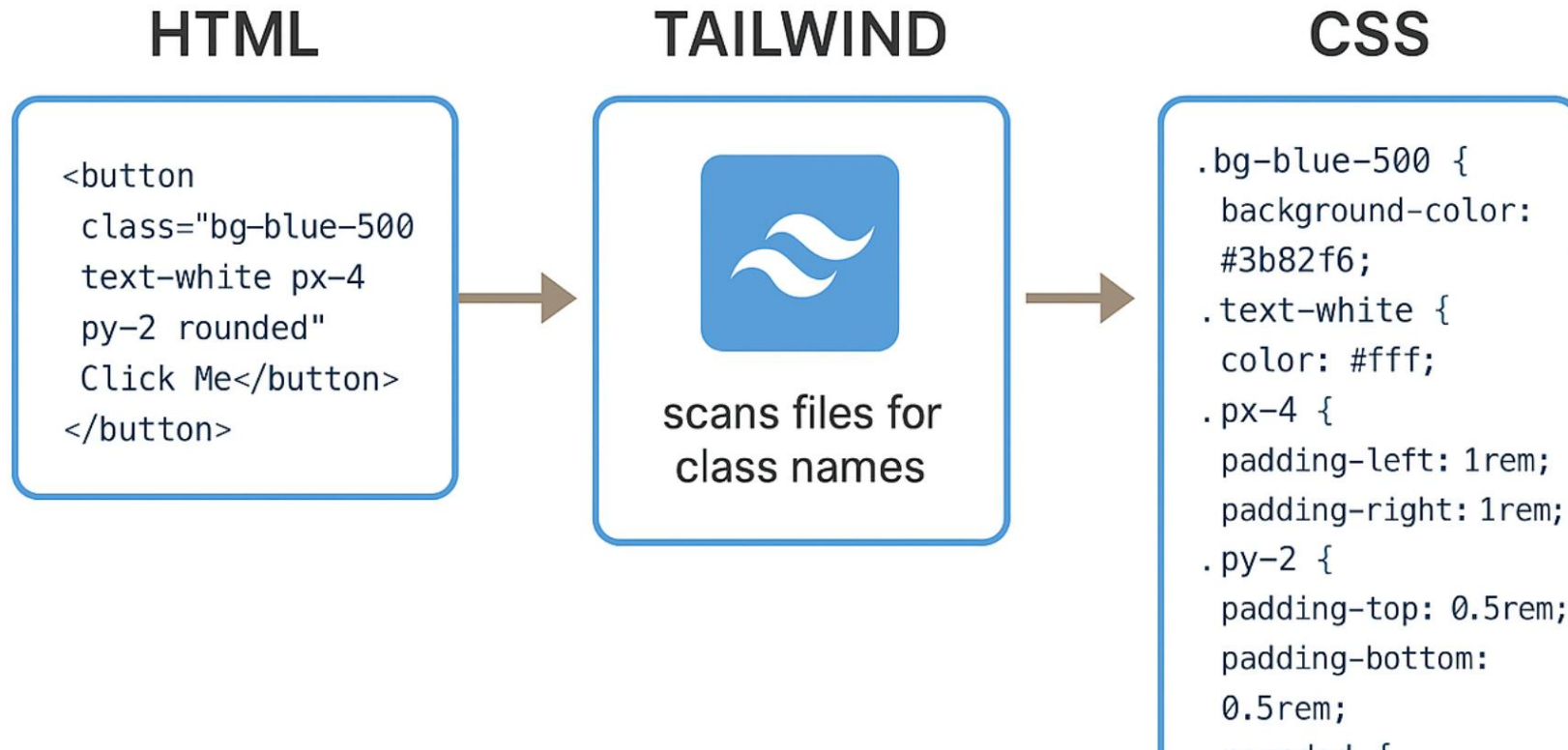
[3] It looks up the corresponding CSS rules from its internal design system.

[4] It then generates a CSS file containing **only** those classes you used.

```
.bg-blue-500 { background-color: #3b82f6; }  
.text-white { color: #fff; }  
.px-4 { padding-left: 1rem; padding-right: 1rem; }  
.py-2 { padding-top: 0.5rem; padding-bottom: 0.5rem; }  
.rounded { border-radius: 0.25rem; }
```

How Utility Classes Help!?

Dr. Mohamed Sami Rakha



Different Ways to Use Tailwind CSS

[1] Use Tailwind via CDN (Quick Start)

Dr. Mohamed Sami Rakha

```
<!DOCTYPE html>
<html lang="en">
<head>
  <script src="https://cdn.tailwindcss.com"></script>
</head>
<body class="bg-gray-100 p-6 text-center">
  <h1 class="text-3xl font-bold text-blue-600">Hello Tailwind!</h1>
</body>
</html>
```

Advantages:

- No installation required
- Perfect for learning or demos

Disadvantages:

- Limited customization (cannot modify theme or add plugins)
- Slower (loads entire library)

[2] Use Tailwind via CLI (Local Build), this is the actual use of Tailwind for Projects **[Must use in the course project]:**

- Generates only the classes you use (smaller CSS)
- Let's you customize colors, fonts, spacing, etc.

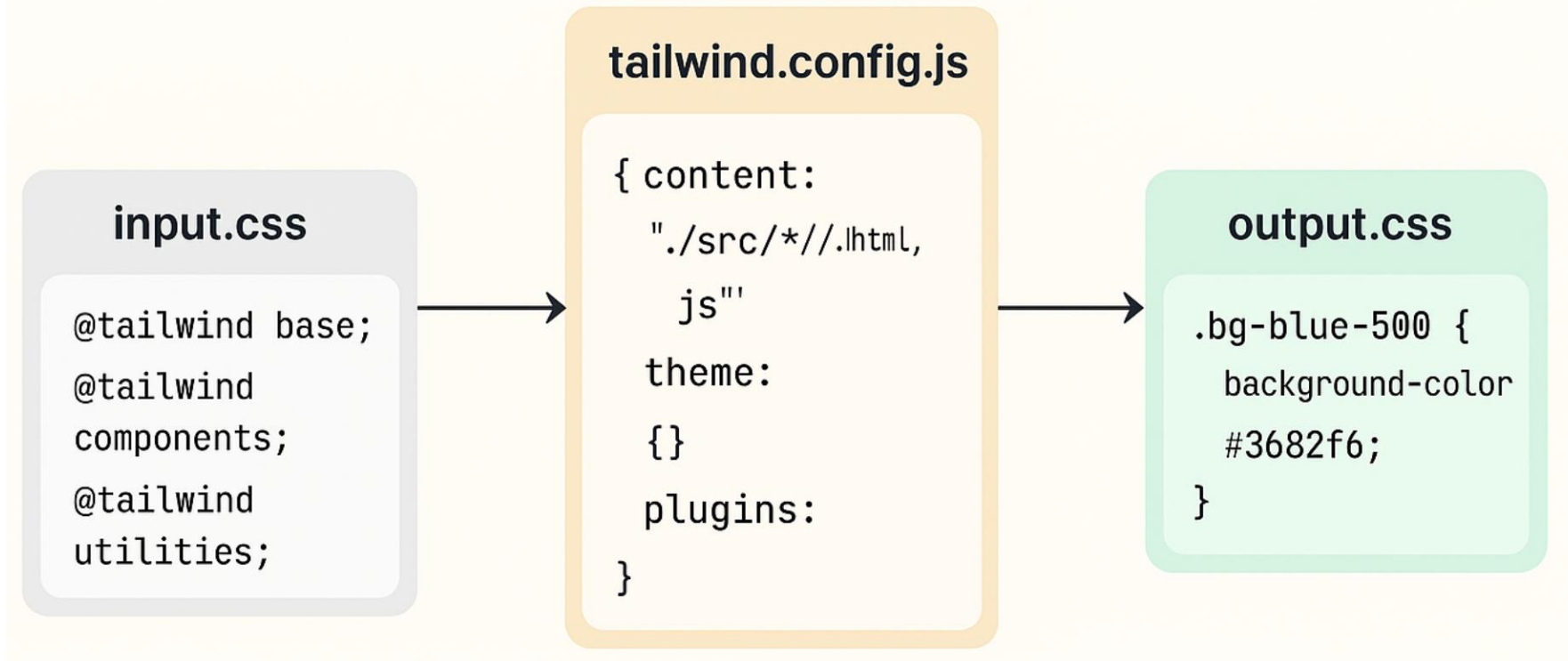
```
npm install -D tailwindcss
npx tailwindcss init
```

npx is a tool that comes with Node.js (via npm)

Tailwind build process order

`npx tailwindcss -i ./src/input.css -o ./dist/output.css --watch` Rakha

The entry point for the Tailwind CLI or build tool is the **“input.css” file.**



`input.css` (instructions) + `tailwind.config.js` (customization) → **processed by Tailwind CLI** → `output.css` (ready-to-use file).

Tailwind build process order

This is your **source CSS file**, it tells Tailwind **what to generate**. It includes **directives** that import **Tailwind's layers**.

input.css

```
@tailwind base;  
@tailwind  
components;  
@tailwind  
utilities;
```

```
@tailwind base;  
@tailwind components;  
@tailwind utilities;
```

@tailwind base; → adds Tailwind's reset styles and base HTML elements.

@tailwind components; → includes predefined reusable classes (like buttons, forms).

@tailwind utilities; → adds all utility classes (like bg-blue-500, p-4, text-center).



You can also write your own custom CSS rules here if you want to override something.

Tailwind build process order

This file is your **Tailwind configuration**; it controls **what's included and how it looks**.

tailwind.config.js

```
{ content:
  './src/**/*.html,
  js'
  theme:
    {}
  plugins:
  }
```

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ['./src/**/*.html,js'], // Files Tailwind scans to remove unused CSS
  theme: {
    extend: {
      colors: {
        brand: '#1D4ED8', // custom color
      },
    },
  },
  plugins: [],
}
```

Main parts:

- **content:** tells Tailwind where to look for class names (HTML, JS, etc.)
- **theme:** customize default colors, fonts, spacing, breakpoints
- **extend:** safely add new styles without losing the defaults
- **plugins:** add extra functionality (like typography, forms, etc.)

Dr. Mohamed Sami Rakha

Thank You...